SmartGitTagger

Automated GitHub repository topic generator

Software Project Lab 3 Technical Report

Submitted by
Sazzad Hossain
BSSE-1207
Institute of Information Technology
University of Dhaka

Supervised by
Dr. Md. Nurul Ahad Tawhid
Associate Professor
Institute of Information Technology
University of Dhaka

Submission Date: 1 December, 2024



Letter of Transmittal

1 December, 2024
BSSE 4th Year Exam Committee
Institute of Information Technology
University of Dhaka

Subject: Technical Report Submission - **SmartGitTagger**: Automated GitHub repository topic generator

Sir,
I am submitting the technical report for our Software Project Lab 3, "**SmartGitTagger**: Automated GitHub repository topic generator". This report comprehensively covers the project's technical details, including design, implementation, and testing phases. While I've tried to do my best, I welcome your feedback for improvement.

Thank you for your consideration.

Sincerely,

Sazzad Hossain
BSSE-1207
Institute of Information Technology
University of Dhaka



Supervisor's Signature

Acknowledgement

I am grateful to the Institute of Information Technology for giving me the opportunity to work on the project "SmartGitTagger: Automated GitHub repository topic generator". I would like to convey my tremendous gratitude to my supervisor, Dr. Md. Nurul Ahad Tawhid, Associate Professor, Institute of Information Technology, University of Dhaka, for providing me with guidance and support throughout the project. His expertise and insights were instrumental in shaping the direction of the project. Lastly, I would like to thank my classmates. They have always been helpful and provided valuable insights from time to time.

Abstract

GitHub is a central platform for developers to host and discover open-source projects. However, the inconsistent tagging of repositories often leads to reduced discoverability and inefficient categorization. SmartGitTagger is an AI-powered Chrome extension designed to address this issue by automatically generating optimized and relevant topics for GitHub repositories. Leveraging a LLM model, the extension analyzes README files and existing metadata to generate meaningful tags.

SmartGitTagger also integrates tagging analysis with popularity ranking to ensure that suggested topics are relevant and widely used. Developers can further customize these tags with real-time feedback on their relevance and impact. With its lightweight design, SmartGitTagger provides fast and accurate results, enhancing repository discoverability and organization on GitHub while offering a seamless user experience through an intuitive interface. By bridging the gap between automation and customization, SmartGitTagger empowers developers to streamline their repository tagging process effectively.

SmartGitTagger not only simplifies the tagging process but also ensures that repositories are accurately categorized, fostering better visibility and collaboration in the open-source ecosystem. As the demand for efficient repository management grows, SmartGitTagger stands as a practical and essential tool for developers seeking to enhance their projects' impact and accessibility.

Table of Content

1. Introduction 6
1.1 Motivation 6
1.2 Scope 7
1.3 Project Timeline 7
1.4 Purpose of Document 7
The purposes of this document are- 7
2. Project Description 8
2.1. Quality Function Deployment 8
2.1.1. Normal Requirements 8
2.1.2. Expected Requirements 8
2.1.3. Exciting Requirements 9
2.2 Usage Scenario 9
Installation 9
Real-time Topic Generation 9
Real-time Topic Suggestions 9
3. Scenario Based Modeling 10
3.1 Use Case Diagram 10
3.1.1 SmartGitTagger 10
3.1.2 Modules of SmartGitTagger 11
3.2.2.1. Trigger Visit Event 11
3.2.2.2. Topic & Report Generation 11
3.2.2.3. Topic Ranking 11
4. Data Based Modeling 12
5. Class-Based Modeling 13
5.1 Analysis Classes 13
5.2 Class cards 14
6. Architectural Design 17
6.1 Architectural Context Diagram 17
6.2 Archetypes 18
6.3 Top Level Components 21
7. Preliminary Test Plan 21
7.1 High Level Description of Testing Goals 21
7.2 Test Cases 22
Conclusion 23
References: 23

List Of Figures

1. Project Timeline(estimated) ……8
2. UCD(level-0)……..11
3. UCD(level-1)……..12
4. ER-diagram……13
5. CRC-diagram……18
6. Architectural Context Diagram ……19
7. Archetypes-0……..20
8. Archetypes-1 ……20
9. Top Level Components ……22

List of Tables

1. Schemas of SmartGitTagger……14
2. CRC Card (Repository) ……..15
3. CRC Card (TopicGenerator) ……..16
4. CRC Card (Model) ……..16
5. CRC Card (TopicRanker) ……17
6. CRC Card (ReportGenerator). ……17

1. Introduction

In the dynamic world of open-source development, GitHub serves as a cornerstone for hosting and discovering repositories, empowering millions of developers globally. A key aspect of repository management on GitHub is the use of topics (or tags) to categorize and enhance the discoverability of projects. However, inconsistent, incomplete, or irrelevant tagging often hinders developers from effectively showcasing their work or finding repositories that align with their needs.

SmartGitTagger steps in as a powerful solution to this challenge. Designed as a seamless Chrome extension, SmartGitTagger leverages advanced AI capabilities, to automatically analyze GitHub README files and suggest optimized topics. By integrating tagging analysis with popularity ranking, the tool ensures developers receive actionable and contextually relevant tag recommendations. Additionally, it allows users to customize tags with real-time feedback, bridging the gap between automation and manual curation.

1.1 Motivation

The motivation behind SmartGitTagger arises from the growing reliance on GitHub as the primary platform for hosting and discovering open-source projects. With over 100 million repositories and a vast ecosystem of developers collaborating on diverse technologies, GitHub has become indispensable to modern software development. However, the sheer volume of repositories also creates a challenge: inconsistent or incomplete tagging.

Inefficient tagging makes it harder for projects to gain visibility and for users to discover relevant repositories. This problem is particularly pronounced in repositories with minimal documentation or ambiguous purposes, where manual tagging may overlook critical keywords or fail to reflect the repository’s scope accurately. As a result, many valuable projects remain underutilized, and developers face difficulties navigating the ecosystem effectively.

SmartGitTagger draws inspiration from advancements in natural language processing and the success of AI models like GPT in understanding and generating human-like text. By analyzing README files—the cornerstone of a repository's documentation—SmartGitTagger aims to automatically generate contextually relevant topics, bridging the gap between automation and human curation. The integration of tagging analysis and popularity ranking ensures that the generated topics align with current trends, enhancing repository discoverability and fostering better collaboration.Driven by the need to streamline repository management and support open-source growth, SmartGitTagger is committed to simplifying the tagging process, reducing inconsistencies, and empowering developers to focus on innovation rather than administrative tasks. By leveraging state-of-the-art AI, it provides a practical and scalable solution to a pressing issue in the modern software development ecosystem.

1.2 Scope

The scope of SmartGitTagger is outlined as follows:

- SmartGitTagger will be a Chrome extension designed to seamlessly integrate with the GitHub platform.
- It will automatically generate relevant topics for GitHub repositories based on their README files:
    - **Automatically**: The extension will detect and suggest topics without requiring manual input from the user.
    - **Fast and Efficient**: Topic generation will occur within seconds, leveraging the power of LLM transformer for rapid text analysis and classification.
- **Topic Popularity Ranking**: The tool will rank generated topics based on their popularity within the GitHub ecosystem, helping prioritize the most relevant tags for discoverability.

1.3 Project Timeline

Tasks	1-15 days	15-20 days	20-50 days	50-90 days	90-110 days
Project Initialization	✓				
Prerequisite knowledge	✓	✓			
SRS documentation			✓		
Project design			✓		
Project development				✓	
Project testing					✓
Report writing					✓

Fig 1: Project Timeline(estimated)

1.4 Purpose of Document

The purposes of this document are-

- Identify and analyze the requirements
- Design the architecture

- Design the test plan
- Describe methodology
- Reduce the development effort
- Improve understanding

2. Project Description

The following depicts the overview of the quality function deployment of SmartGitTagger. The journey towards defining the project's Quality Function Deployment (QFD) began with an in-depth review of relevant research literature, notably the paper authored by Juri Di Rocco1 , Davide Di Ruscio1 [1], providing valuable insights into the domain. In addition to the academic research, stakeholder engagement played a pivotal role in shaping the scope of the project. This involved discussions with developers, repository managers, as well as open-source contributors, who shared their experiences with GitHub's tagging system and provided feedback on potential improvements. Their input highlighted the need for a tool that could streamline the tagging process, reduce inconsistencies, and increase repository visibility. As a result, SmartGitTagger was conceptualized to address these challenges by using AI to automatically generate relevant topics from README files, enhancing both the user experience and repository discoverability on GitHub.

2.1. Quality Function Deployment

Quality Function Deployment (QFD) is a technique that translates the needs of the customer into technical requirements for the software. With respect to this project, the following requirements are identified

2.1.1. Normal Requirements

1. **Chrome Extension Functionality:**
   SmartGitTagger should successfully analyze GitHub repository README files using LLM transformer to automatically generate relevant and optimized topics based on the content, ensuring accurate categorization.
2. **Efficiency:**The extension should process repository analysis and topic generation swiftly, ensuring minimal impact on user experience, with results provided in a few seconds.

2.1.2. Expected Requirements

1. **Topic Relevance and Accuracy:**
   SmartGitTagger's AI model should demonstrate a high level of accuracy in generating relevant and popular topics for each repository, reflecting current trends and use cases on GitHub.

2. **Ease of Installation and Integration:**
   Installing and configuring the extension should be easy and user-friendly, with clear instructions for setting up and using the extension within Chrome.

2.1.3. Exciting Requirements

1. **Topic Generation Reports:**
   It should provide a basic summary report of generated topics, including verdicts (approved, recommended, or irrelevant topics) and key indicators (e.g., topic relevance and popularity).
2. **Topic Popularity Ranking:**
   Beyond simply generating topics, the extension could suggest popular or trending topics based on current GitHub trends, offering users optimized tags for increased repository discoverability.

2.2 Usage Scenario

SmartGitTagger: Optimized Topic Generator for GitHub Repositories" will be a Chrome extension designed to help users automatically generate relevant topics for GitHub repositories based on their README files. SmartGitTagger will provide real-time analysis and deliver actionable topic suggestions, improving repository discoverability and categorization. The usage scenario for the extension can be segmented as follows:

Installation

Users can download and install the SmartGitTagger extension directly from the Chrome Web Store. Once installed, it integrates seamlessly with the GitHub interface and begins analyzing repositories automatically.

Real-time Topic Generation

As users browse GitHub repositories, SmartGitTagger will automatically evaluate the repository's README content. It will use LLM transformer to generate optimized topics by analyzing the repository's description, technologies used, and the existing metadata like programming language and current tags.

Real-time Topic Suggestions

When a user opens a repository, SmartGitTagger will display suggested topics directly within the browser interface. If the suggested topics are relevant, they will appear alongside the existing tags, allowing the user to review and modify them. If the generated topics are less relevant, the user can manually adjust them.

3. Scenario Based Modeling

Scenario-based modeling is an approach used in software development and design to create models that depict how a system behaves or reacts in various real-world situations or scenarios. These scenarios are typically based on user interactions, events, or conditions and help developers understand and visualize how the software will function in different contexts. By simulating these scenarios, developers can better identify potential issues, make informed design decisions, and ensure that the software meets user requirements across different usage contexts.

3.1 Use Case Diagram

A use case diagram is a visual representation in software engineering that illustrates how a system interacts with its external entities or actors to achieve specific goals or tasks. It shows the relationships between use cases (representing system functionalities) and actors (representing external users or systems). Use case diagrams are typically created through requirements gathering, where system functionalities and user interactions are identified, and then diagrammed using specialized software modeling tools.

3.1.1 SmartGitTagger

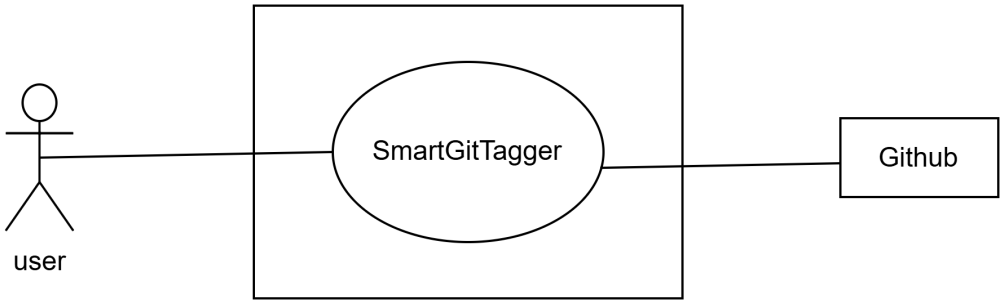


Figure-2: Level 0

Primary actors: User

Secondary actors: Github

3.1.2 Modules of SmartGitTagger

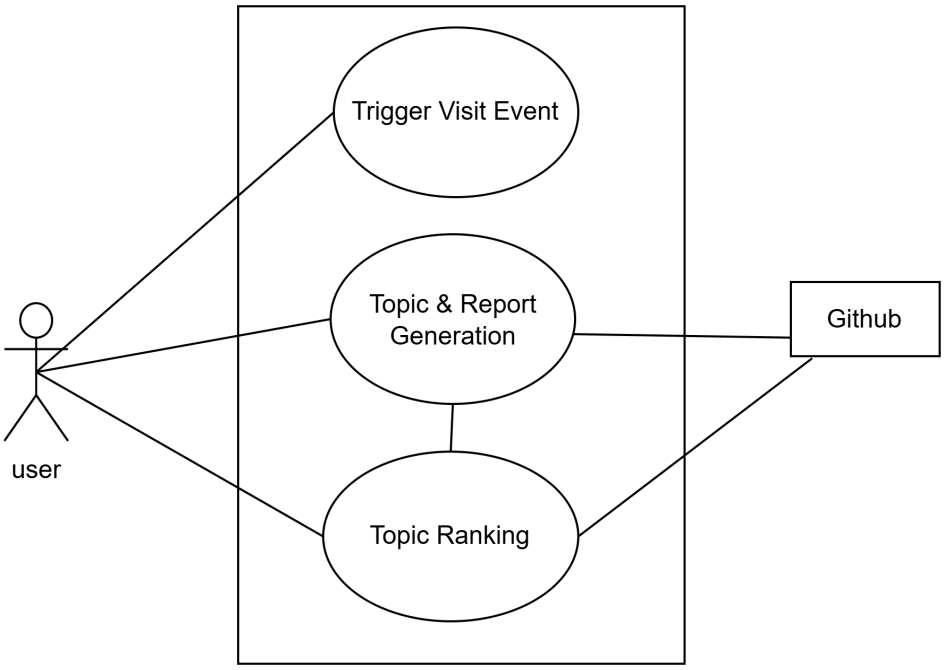


Figure-3: UseCase Level 1 (Modules of SmartGitTagger)

Primary Actor: User
Secondary Actor: Github

3.2.2.1. Trigger Visit Event

The "Trigger Visit Event" module serves as the entry point for SmartGitTagger functionality. It listens for user interactions and initiates the analysis process when users visit the github repository url. This module is responsible for activating SmartGitTagger's topic generation process.

3.2.2.2. Topic & Report Generation

The Topic & Report Generation module is the core functionality of SmartGitTagger. Once triggered by the Visit Event, this module analyzes the content of the GitHub repository's README file using an LLM transformer. The module processes the README text, existing tags, and metadata (e.g., stars, forks, and language) to generate a set of relevant, optimized topics. It also generates a relevant briefing about the generated topics. These topics aim to enhance the repository's discoverability and categorization on GitHub. The generated tags are then displayed to the user for review and potential customization.

3.2.2.3. Topic Ranking

The Topic Ranking module is a crucial enhancement of the SmartGitTagger functionality. After

generating a set of topics from the README file, this module evaluates and ranks the topics based on their popularity within the GitHub ecosystem. It processes metrics such as the number of repositories tagged with a given topic, associated star and fork counts, and recent activity trends.

4. Data Based Modeling

Database modeling is a visual representation of a database structure and how data is organized within it. It involves creating entity-relationship diagrams (ERDs) that depict tables (entities), attributes (columns), and the relationships between them. Database modeling is typically achieved by analyzing the data requirements of a system, identifying entities and their attributes, and then using modeling tools or software to create ERDs that serve as a blueprint for database design and implementation.

The entities of SmartGitTagger after analyzing the usage story are:

- Repositories
- Topics

The ER-Diagram of SmartGitTagger can be shown as below:

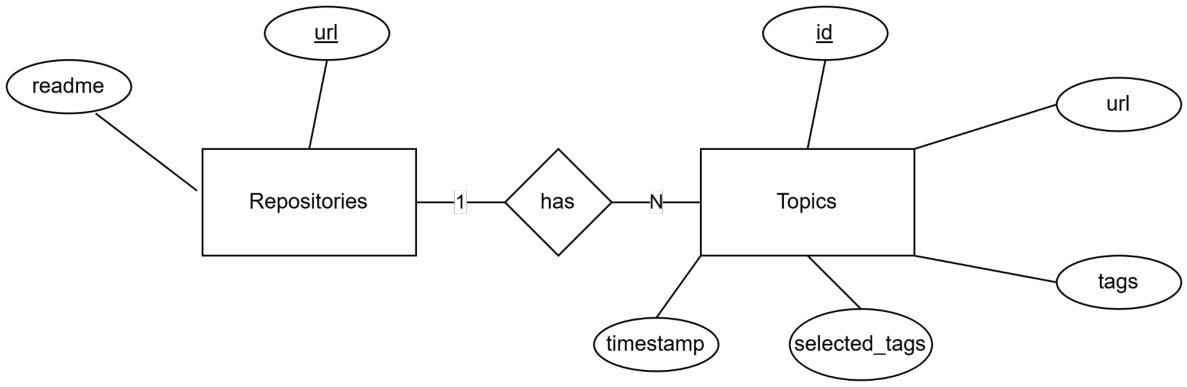


Figure-4: ER-Diagram of SmartGitTagger

The schemas derived from the ER-diagram is as follows:

Schema	Attributes	Type
Repositories	url(PK)	varchar(100)
	readme	varchar(1000)
Topics	id(PK)	int
	url(FK)	varchar(100)
	tags	varchar(50)[ ]
	selected_tags	varchar(50)[ ]
	timestamp	date

Table-1: Schemas of SmartGitTagger

5. Class-Based Modeling

Class-based modeling is a visual representation in software engineering that illustrates the structure of a software system using classes, their attributes, and relationships. It is commonly used in object-oriented design and modeling.

5.1 Analysis Classes

After identifying nouns from the scenario, I have filtered nouns belonging to the solution domain using General Classification (External entities, Things, Events, Roles, Organizational units, Places, and Structures). Nouns selected as a potential class were filtered using Selection Criteria (Retained information, Needed services, Multiple attributes, Common attributes, Common operations, and Essential requirements). After performing analysis on potential classes, I have found the following analysis classes -

- Repository
- TopicGenerator
- Model
- TopicRanker
- ReportGenerator

5.2 Class cards

Repository	
Attributes	**Methods**
-repositoryURL -readme -existingTopics	+validateURL() +fetchReadme()
Responsibilities	**Collaborators**
● Manage repository information such as README. ● Provides data for topic generation and recommendations.	TopicGenerator

Table-2: CRC Card (Repository)

TopicGenerator	
Attributes	**Methods**
-model -generatedTopics	+generateTopics()
Responsibilities	**Collaborators**
● Analyze README to generate topics using the LLM transformer. ● Ensure topics are optimized and contextually relevant.	Repository, TopicRanker, model

Table-3: CRC Card (TopicGenerator)

Model	
Attributes	**Methods**
-modelName -apiEndpoint -generationStrategy	+extractTopics()
Responsibilities	**Collaborators**
• Analyze README to generate topics using the LLM transformer. • Ensure topics are optimized and contextually relevant.	TopicGenerator

Table-4: CRC Card (Model)

TopicRanker	
Attributes	**Methods**
-repositoryCount -starCount -forkCount -trendingTopics	-calculateWeight() +rankTopics()
Responsibilities	**Collaborators**
• Provide ranked topics for better discoverability.	Repository, TopicGenerator

Table-5: CRC Card (TopicRanker)

ReportGenerator	
Attributes	**Methods**
-reportContent	+generateReport()
Responsibilities	**Collaborators**
• Categorize generated, existing and used topics in the repository	TopicGenerator, TopicRanker

Table-6: CRC Card (ReportGenerator)

The CRC diagram of the classes can is given below:

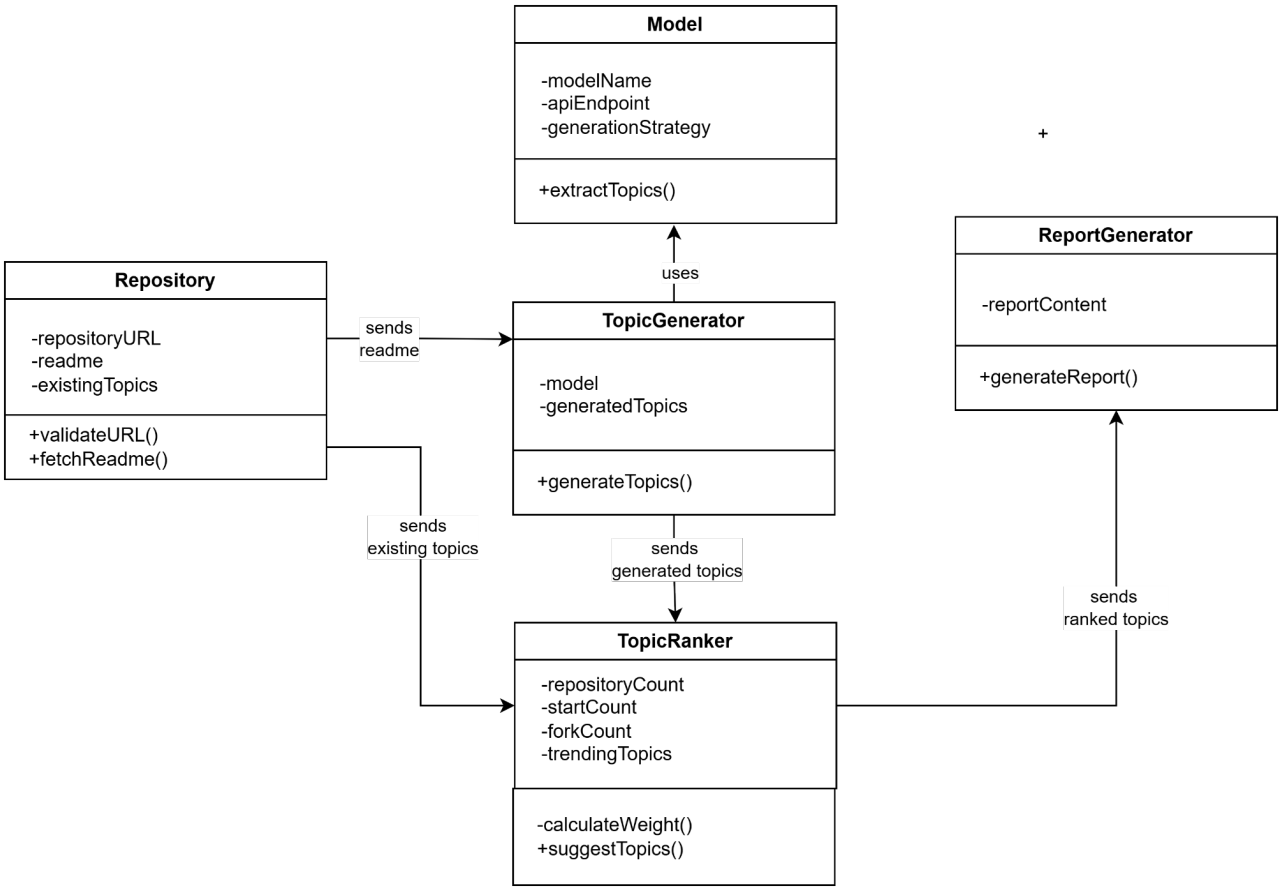


Figure-5: CRC Diagram of SmartGitTagger

6. Architectural Design

Architectural design is a visual representation in software engineering that outlines the high-level structure and organization of a software system. It focuses on defining the major components or modules of the system, their interactions, and the overall system's architecture.

6.1 Architectural Context Diagram

The architectural context diagram for the tool is given below -

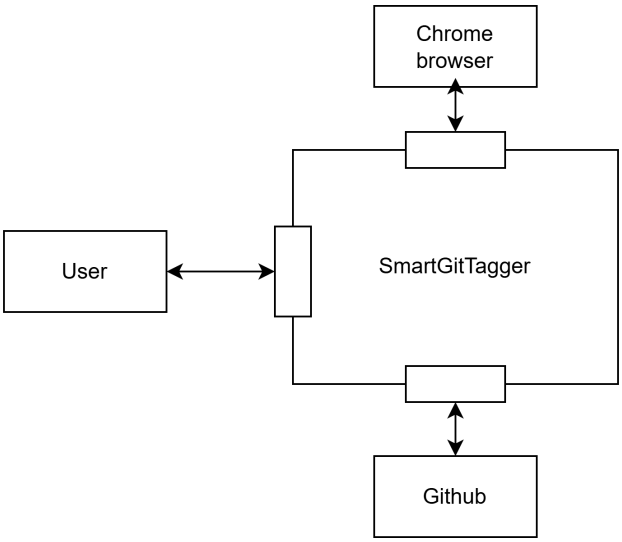


Figure-6: Architectural Context Diagram

One category of user is supposed to be the target for the tool. They are owners of github repositories. There is one subordinate system, github. The system is the source of input data to the system. And Chrome browser will be the only superordinate system of SmartGitTagger as they will be using this system as extension. However there is no peer-level system.

6.2 Archetypes

The archetypes for the tool are defined below -

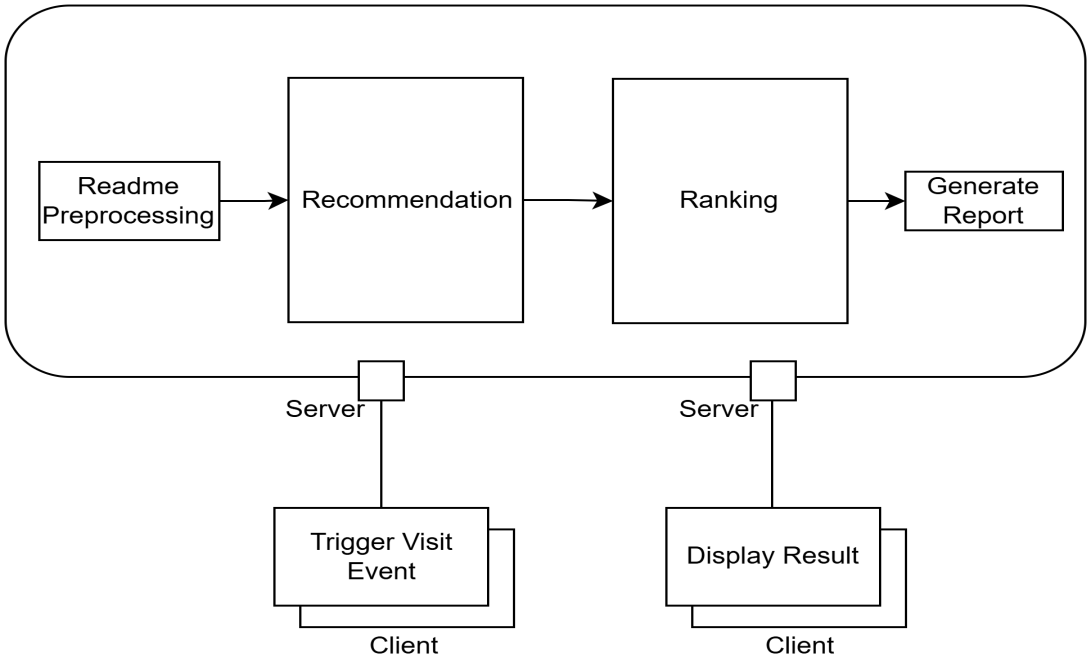


Figure-7: SmartGitTagger (Client-Server)

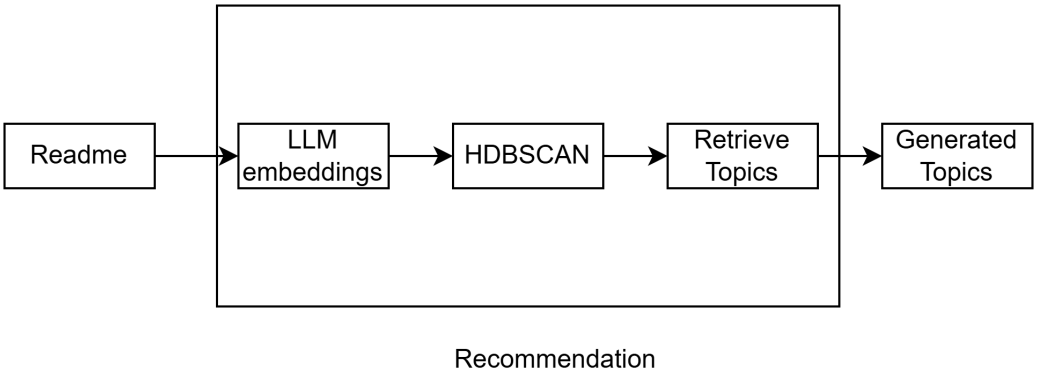


Figure-8: Topic Recommendation (Pipes and Filter)

- **Trigger visit event:** This segment initiates the process by triggering a user's visit to github repository.

- **Readme Preprocessing:** This segment is responsible for preprocessing the readme file data and tokenize it for the next step .

- **Recommendation:**
  - **LLM embeddings:** This segment generates embeddings from preprocessed readme using fine-tuned LLM transformers.
  - **HDBSCAN:** Next,this segment creates suitable clusters of similar words on the generated embeddings.
  - **HDBSCAN:** Lastly,this segment retrieves the best possible topic from each cluster.

- **Refinement**:
  - **Topic Filtering:** This segment refines suitable words combining with existing and recommended topic words from the previous step.

  - **Generate report:** This segment compiles the recommended, existing, used and unused topics for each repository after using this chrome extension.

6.3 Top Level Components

Overall architectural structure with top level components is illustrated below-

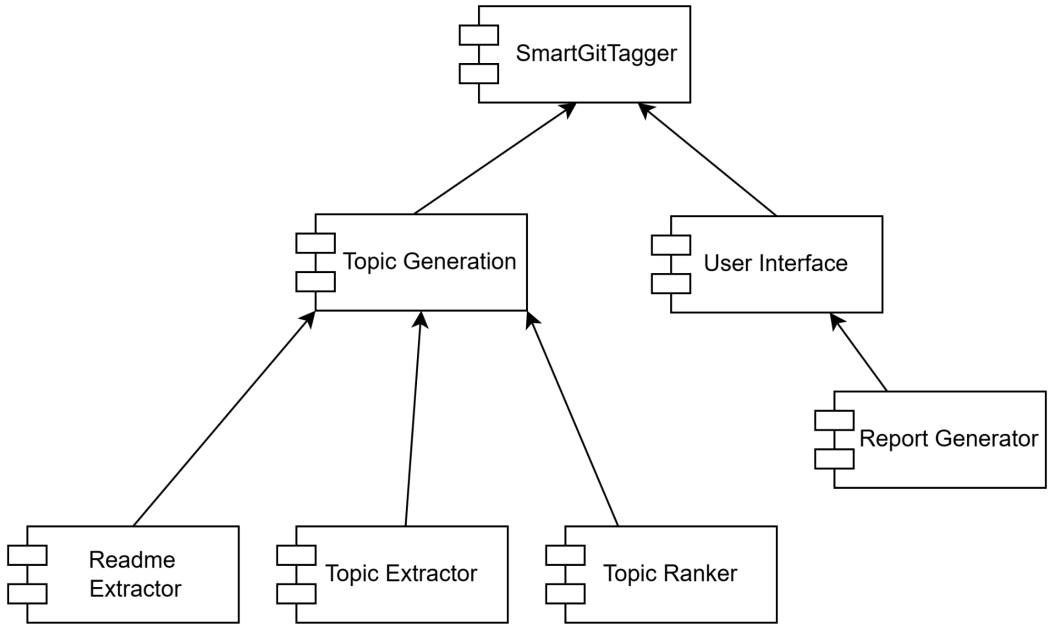


Figure-9: Top Level Components

7. Preliminary Test Plan

In this chapter, a high level description of testing goals and summary of features to be tested are presented.

7.1 High Level Description of Testing Goals

Testing goals for this extension is represented below from a high level-

- To demonstrate that the extension meets its requirements
- To ensure the metadata of the github repository is correctly extracted
- To ensure that it recommends correct topics most of the time
- To ensure that it provides detailed topic report
- To ensure that the tool runs smoothly

7.2 Test Cases

Test cases for the testing of this tool are documented below-

- Test Case 1: Installation and Activation

  Test Scenario:

  - Install SmartGitTagger extension in a fresh Chrome browser.
  - Activate SmartGitTagger.

  Expected Outcome:

  - The extension installs without errors.
  - SmartGitTagger is activated successfully.

- Test Case 2: Extract Readme

  Test Scenario:

  - Open a github repository.
  - Let SmartGitTagger extract Readme from the repository using repo URL.

  Expected Outcome:

  - SmartGitTagger correctly extracts Readme.

- Test Case 3: Topic Words Generation

  Test Scenario:

  - Click the “SmartGitTagger” extension from the “extensions” option of chrome browser.
  - Let SmartGitTagger display the generated topics in a pop-up display.

  Expected Outcome:

  - SmartGitTagger correctly opens up a pop-up window with the recommended topics for the repository.

- Test Case 4: Detailed Topic Report

  Test Scenario:

  - After clicking the “Finish” button on the pop-up extension containing recommended words, it will generate a report with the count of existing, recommended, used and featured topics with respect to corresponding repository.

  Expected Outcome:

  - SmartGitTagger provides a detailed report with information about varieties of topics regarding repository.
  - The report is clear and informative.

Conclusion

In this report, I have discussed all the necessary technical details to develop the “SmartGitTagger: *Automated GitHub repository topic generator*”. I have discussed the scope, assumptions, QFD and usage scenario. Scenario-based modeling, data modeling and class-based modeling have been performed. The architectural design has also been discussed thoroughly. I have also presented the high-level description of testing goals and summary of items and features to be tested. I have added figures and tables for better understanding of the technical details.

Though SmartGitTagger addresses the suitable topics related to github repositories, there are chances of further improvements. Addition of newer language support and other reinforcement learning techniques can be the future scope of SmartGitTagger.
Throughout this document, I have tried to clearly state the workflow of the total system. I hope this report can help the reader have an appropriate understanding of the technical details of this extension tool.

References:

[1] Rocco, Juri & Di Ruscio, Davide & Di Sipio, Claudio & Nguyen, Phuong & Rubei, Riccardo. (2022). HybridRec: A recommender system for tagging GitHub repositories. Applied Intelligence. 53. 10.1007/s10489-022-03864-y.